Digital Design

RTL Design: Sequential Multiplier

A. Sahu
Dept of Comp. Sc. & Engg.
Indian Institute of Technology Guwahati
http://jatinga.iitg.ac.in/~asahu/cs221/

Outline

- Classic Example : Sequential Multiplier
- Verilog Code for Multiplier
- FPGA: Introduction

RTL/ASM Classic example : 8 bit Sequential Multiplier With Verilog Code

Ref: Mano Book

RTL: Multiplier Example

- Example: (101 x 011)
- Partial product sum for *n* digits
  - requires adding up to *n* digitsb (with carries) in a column.
- *n* x *m* digit multiply generates
  - up to an *m + n* digit result

- **Partial products are: 101 x 0, 101 x 1, and 101 x 1**

		1	**0**	**1**	
	×	**0**	**1**	**1**	
		1	**0**	**1**	
	1	**0**	**1**		
0	**0**	**0**			
0	**0**	**1**	**1**	**1**	**1**

Example (1 0 1) x (0 1 1) Again

- Reorganizing example to follow hardware algorithm:

		1	0	1			
	x	0	1	1			
→	0	0	0	0			Clear C || A
→	+	1	0	1			Multipler0 = 1 => Add B
→	0	1	0	1			Addition
→	0	0	1	0	1		Shift Right (Zero-fill C)
→	+	1	0	1			Multipler1 = 1 => Add B
→	0	1	1	1	1		Addition
→	0	0	1	1	1		Shift Right
→	0	0	0	1	1	1	Multipler2 = 0 => No Add, Shift Right

Binary Multiplication

- Polynomial Multiplication
  - You can think Binary number A, B as polynomials
  - $A(X)=A_{n-1}.2^{n-1}+....+ A_2.2^2+ A_1.2^1+ A_0.2^0$
  - Multiply polynomial to get another one
  - Time complexity: $O(n^2)$ Basic serial Algorithm, $O(n^{1.5})$ for divide conquer approach, O(nlgn) using FFT
- Booth Algorithm reduce number Partial addition
  - 99 Represent using 100-1, 95 is 100-5
  - 111 represent as 1000-1 in binary: possibly reduce number addition/sub

Booth's Encoding

- Recall grade school trick
  - When multiplying by 9:
    - Multiply by 10 (easy, just shift digits left), Subtract once
  - E.g. 123454 x 9 = 123454 x (10 – 1) = 1234540 – 123454
- Booth's algorithm applies same principle
  - in binary, just '1' and '0'

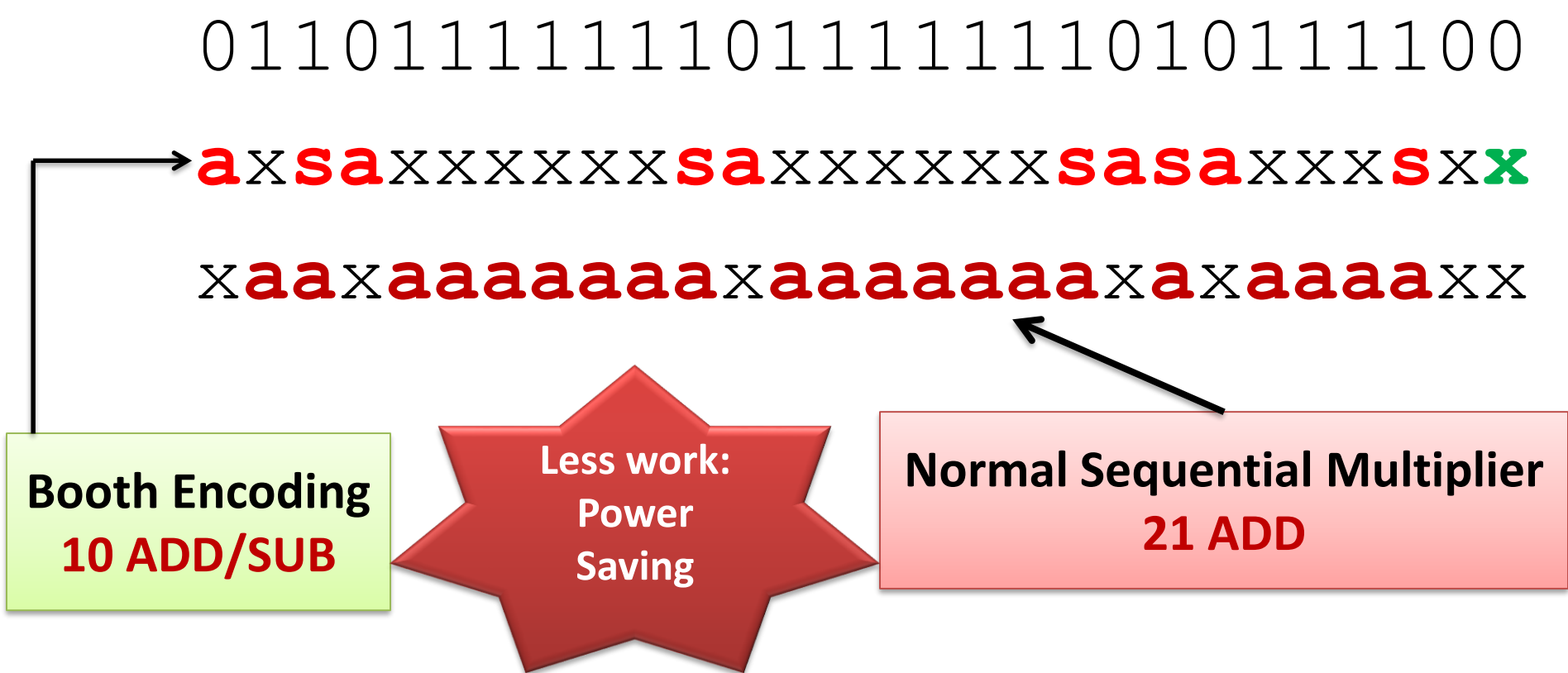
Booth's Encoding

- Search for a run of '1' bits in the multiplier
  - E.g. '01110' has a run of 3 '1' bits in the middle
  - Multiplying by '01110' (14 in decimal) is equivalent to multiplying by 8 and subtracting twice, since 14 x m = (16 – 2) x m = 16m – 2m
  - Convert to two shifts and one subtraction
- Hence, iterate right to left and:
  - Subtract multiplicand from product at first '1'
  - Add multiplicand to product after last '1'
  - Don't do either for '1' bits in the middle

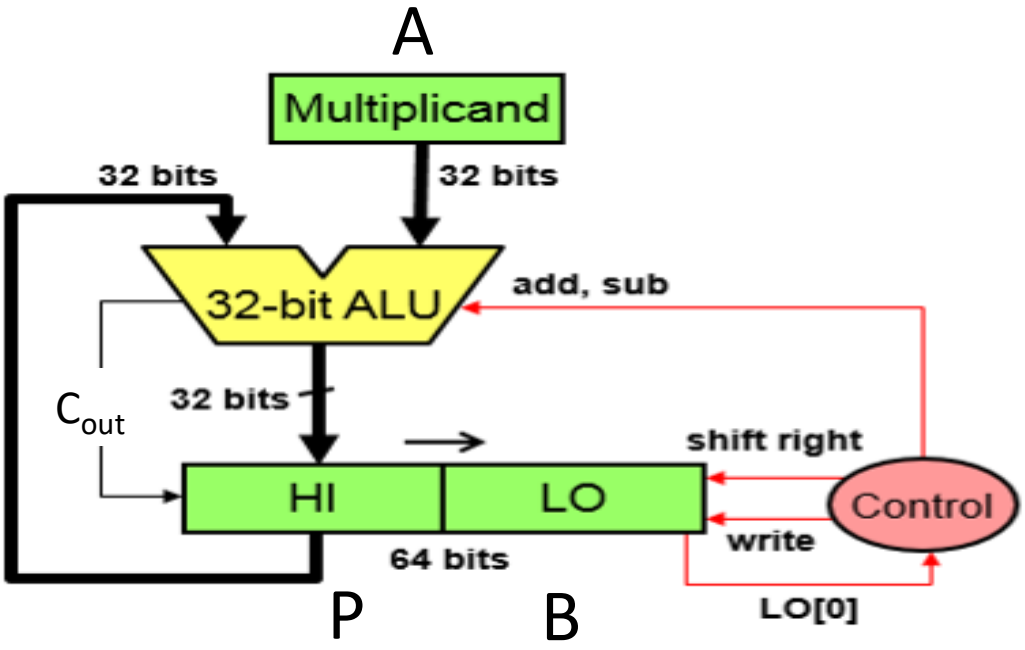
Booth's Algorithm

Current bit	Bit to right	Explanation	Example	Operation
1	0	Begins run of '1'	00001111000	Subtract
1	1	Middle of run of '1'	00001111000	**Nothing**
0	1	End of a run of '1'	00001111000	Add
0	0	Middle of a run of '0'	00001111000	**Nothing**

Booth's Encoding Example



Sequential Multiplier: High Level Algorithm

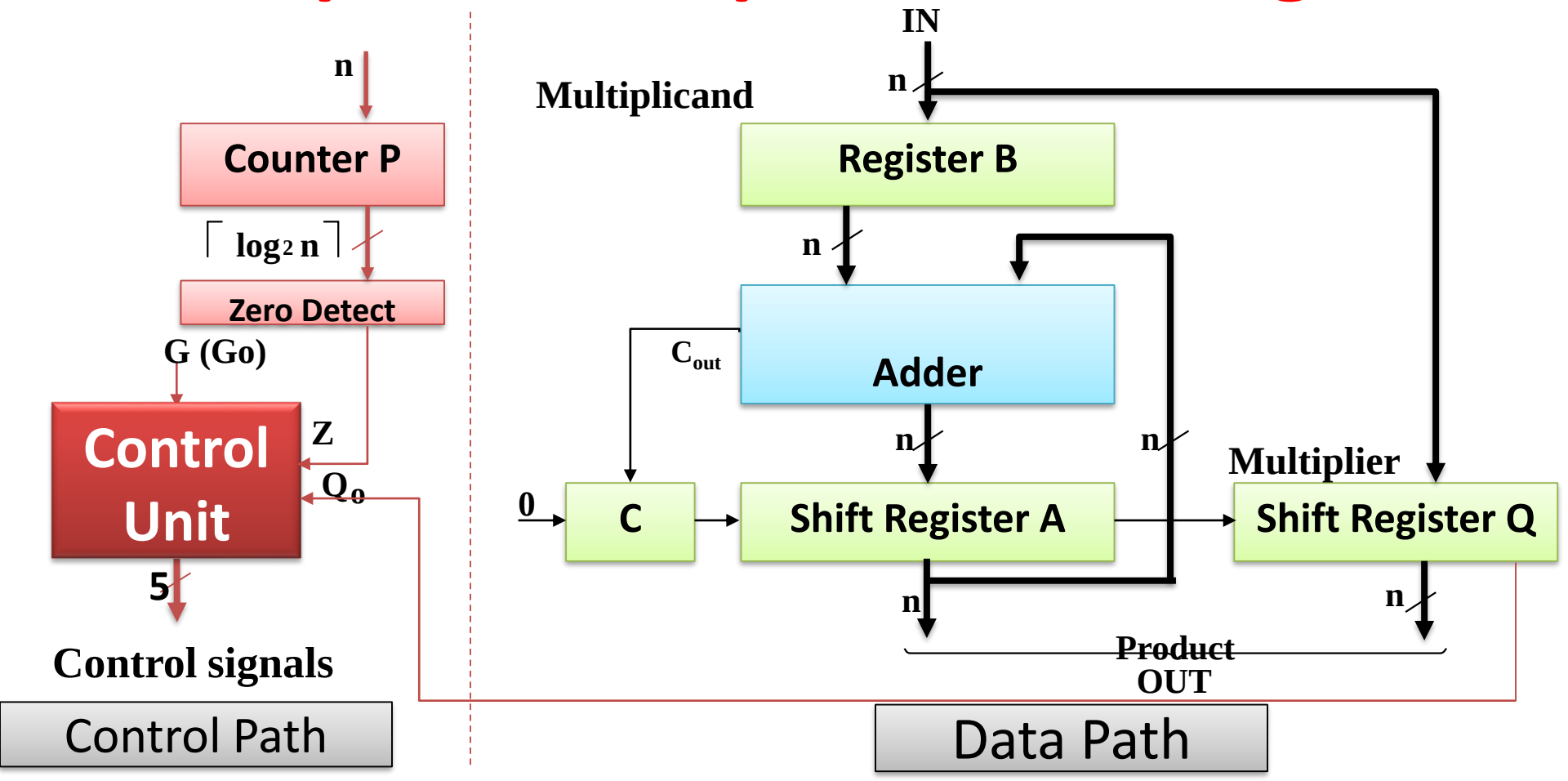


Control Algorithm:

1. P ← 0, A ← multiplicand, B ← multiplier **//Initialization**
2. **If** (LSB of B==1) add A to P **else** add 0
3. Shift [P][B] right 1
4. Repeat steps 2 and 3 for n-1 times.
5. [P][B] has product.

Ref book: Comp. Organization and Design , David A Paterson, 3rd Ed, 2010

Multiplier Example: Block Diagram



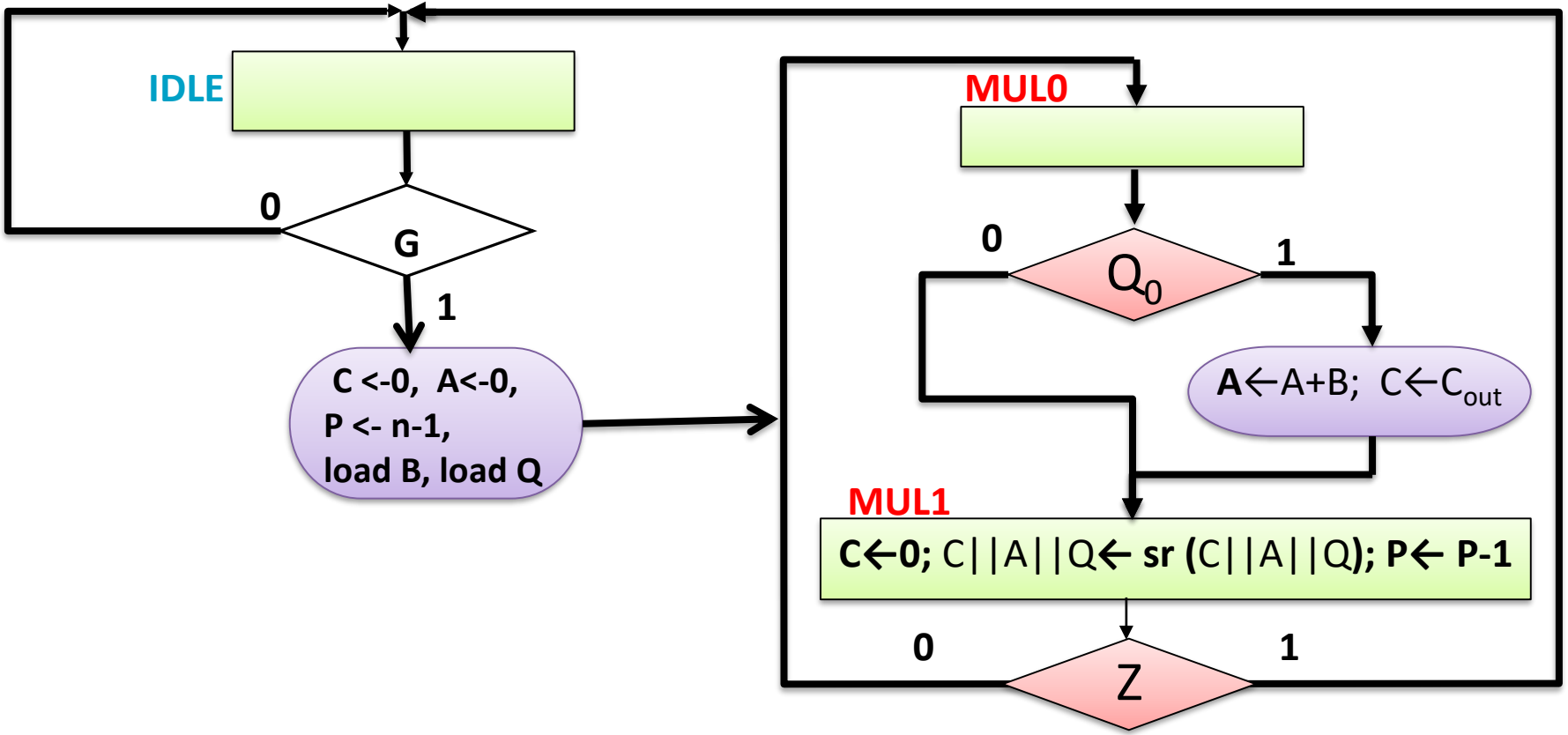
Multiplier Example: Operation

- Step1: The multiplicand (top operand) is loaded into register B.
- Step2: The multiplier (bottom operand) is loaded into register Q.
- Step3: Register C||A is initialized to 0 when G becomes 1.
- Step4: The partial products are summed iteratively in register C||A||Q.

Multiplier Example: Operation

- Step5: Each multiplier bit, beginning with the LSB, is processed (if bit is 1, use adder to add B to partial product; if bit is 0, do nothing)
- Step6: C||A||Q is shifted right using the shift register
  - Partial product bits fill vacant locations in Q as multiplier is shifted out
  - If overflow during addition, the outgoing carry is recovered from C during the right shift
- Step 7: Steps 5 and 6 are repeated until Ctr P = 0
  - Counter P is initialized in step 4 to n – 1, n = number of bits in multiplier

Multiplier Example: ASM Chart



Multiplier Example: ASM Chart (continued)

- Combined Mealy - Moore output model
- **IDLE - state**
  - Input G is used as the condition for starting the multiplication, and
  - *C, A, and P are initialized and Load B, Load Q*
- **MUL0 - state**
  - Conditional addition is performed based on the value of Q_0.
- **MUL1 - state**
  - Right shift is performed to capture the partial product and position the next bit of the multiplier in Q_0
  - the terminal count of 0 for down counter P is used to sense completion or continuation of the multiply.

Multiplier Example: Control Signal Table

Module	Micro Operations	Control Signal Name	Control Expression
Reg A	A <- 0 A <- A +B C ||A||Q <- sr (C ||A||Q)	Load_regs Add_regs Shift CAQ	IDLE.G MUL0.Q_0 MUL1
Reg B	B <- IN	Load_regs	LOAD_B
FF C Reg Q	C <- 0 C <- Cout Q <- IN C ||A||Q <- sr (C ||A||Q)	Clear C Load Load_regs ShiftCAQ	IDLE.G+MUL1 -- LOAD_Q
Ctr P	P <- N-1 P <- P-1	Load_regs Decr_P	 MUL1

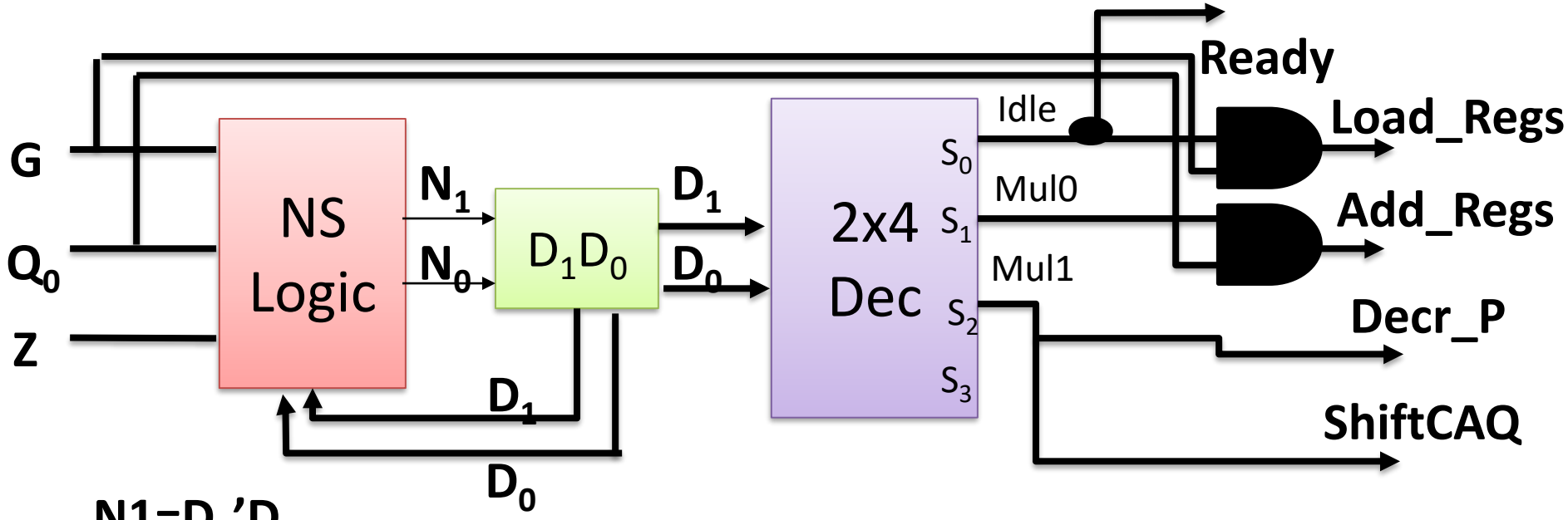
Multiplier Example: Control Circuit

PS	PS	Inputs			NS	Ready	Load Regs	Decr_P	Add_regs	Shift_CAQ
		G	Q0	Z						
Idle	00	0	x	X	00	1				
Idle	00	1	X	x	01	1	1			
Mul0	01	x	0	X	10			1		
Mul0	01	x	1	X	10				1	
Mul1	10	x	x	0	01			1		1
Mul1	10	x	x	1	00			1		1

Ready=Idle Load_Regs=Idle.G Decr_P=Mul0

Add_regs=Mul0.Q_0 ShiftCAQ=Mul1

Multiplier Example: Control Circuit



$N1 = D_1'D_0$

$N0 = D_1'D_0'G + D_1D_0'Z'$

Ready=Idle Load_Regs=Idle.G Decr_P=Mul1

Add_regs=Mul0.Q_0 ShiftCAQ=Mul1

Verilog Code For Multiplier

```
//IO Description
module Seq_Bin_Mult(Product, Ready, Multiplicand, Multiplier,
Start,  clock, reset_b);
        parameter dp_width = 5; // Set to width of datapath
        parameter BC_size = 3; // Size of bit counter

        output [2*dp_width -1: 0] Product;
        output Ready;
        input [dp_width -1: 0] Multiplicand, Multiplier;
        input Start, clock, reset_b;
```

Verilog Code For Multiplier

```
//ASM State One Hot Encoding
parameter S_idle=3'b001, S_add= 3'b010, S_shift=3'b100;
        reg [2: 0] state, next_state;
//Data Path Components
        reg [dp_width -1: 0] A, B, Q; // Sized for datapath
        reg C;
        reg [BC_size -1: 0] P;
        reg Load_regs, Decr_P, Add_regs, Shift_regs;
// Miscellaneous combinational logic
assign Product = {A, Q};
wire Zero = (P == 0); // counter is zero
wire Ready = (state == S_idle); // controller status
```

Verilog Code For Multiplier

```
always @ (posedge clock, negedge reset_b) // control unit
  if (~reset_b) state <= S_idle; else state <= next_state;
     always @ (state, Start, Q[0], Zero) begin
       next_state=S_idle;
      Load_regs=0; Decr_P=0; Add_regs=0;Shift_regs = 0;
     case (state)
            S_idle: begin if (Start) next_state=S_add; Load_regs=1; end
            S_add: begin next_state = S_shift; Decr_P = 1;
                        if (Q[0]) Add_regs = 1; end
            S_shift: begin Shift_regs = 1; if (Zero) next_state = S_idle;
                                    else next_state = S_add; end
            default : next_state = S_idle;
    endcase
end
```

Verilog Code For Multiplier

```
// datapath unit
always @ (posedge clock) begin
        if (Load_regs) begin
                P <= dp_width;  A <= 0;  C <= 0;
                B <= Multiplicand; Q <= Multiplier;
                end
        if (Add_regs) {C, A} <= A + B;
        if (Shift_regs) {C, A, Q} <= {C, A, Q} >> 1;
        if (Decr_P) P <= P -1;
end
endmodule
```

Verilog Code For Multiplier : Test1

```
// Test bench for the binary multiplier
module tb_Seq_Bin_Mult;
        parameter dp_width = 5; // Set to width of datapath
        wire [2*dp_width -1: 0] Product; // Output from multiplier
        wire Ready;
        reg [dp_width -1: 0] Multiplicand, Multiplier; // Inputs
        reg Start, clock, reset_b;
        // Instantiate multiplier
        Seq_Bin_Mult M0 (Product, Ready, Multiplicand, Multiplier,
                         Start, clock,reset_b);
        // Generate stimulus waveforms
        initial #200 $finish ;
```

Verilog Code For Multiplier: Test1

```
Initial begin
        Start = 0;
        reset_b = 0;  #2 Start = 1; reset_b = 1;
        Multiplicand= 5'b10111; Multiplier = 5'b10011; //23*19=437
        #10 Start = 0;
end
Initial begin  clock = 0;  repeat (26) #5 clock = ~clock;  end
always @ (posedge clock) // Display results
        $strobe ("C=%b A=%b Q=%b P=%b  time=%0d",
                                M0.C,M0.A,M0.Q,M0.P, $time);
endmodule
```

Verilog Code For Multiplier: Result 1

```
$ iverilog SeqMult.v
$./a.out
C=0 A=00000 Q=10011 P=101 time=5
C=0 A=10111 Q=10011 P=100 time=15
C=0 A=01011 Q=11001 P=100 time=25
C=1 A=00010 Q=11001 P=011 time=35
C=0 A=10001 Q=01100 P=011 time=45
C=0 A=10001 Q=01100 P=010 time=55
C=0 A=01000 Q=10110 P=010 time=65
C=0 A=01000 Q=10110 P=001 time=75
C=0 A=00100 Q=01011 P=001 time=85
C=0 A=11011 Q=01011 P=000 time=95
C=0 A=01101 Q=10101 P=000 time=105
```

//Multiplying 10111 (23) with 10011 (19) will result 110110101 (437)

//result is
CAQ= 0 **01101 10101**

Vector Dot Product Unit: VDPU

- Vector Dot Product
- Common for Matrix Mul, Neural Network, CNN, DNN and Many machine Learning
- S = $\sum_{i=0}^{N-1} W_i X_i$
- Each cycle one Multiplication and Addition

Verilog Code For VDPU

```
module VDPU (
    input clk,  input reset,
    input [7:0] num1,
    input [7:0] num2,
    output reg [15:0] result
);

reg [15:0] temp_result; // result
reg [2:0] count; // counter for multiplication
```

Verilog Code For VDPU

```
always @(posedge clk or posedge reset) begin
   if (reset) begin
      result <= 16'd0;    temp_result <= 16'd0;     count <= 3'd0;
   end
   else begin    if (count < 3'd7) begin
                 temp_result <= num1 * num2;
                 result <= result + temp_result;
                 count <= count + 1;
                 end
        end
   end
endmodule
```

Thanks